

# SVFinder: User Manual & System Setup Guide

---

This guide provides a comprehensive roadmap for configuring the local environment, installing structural dependencies, executing the data harvesting pipelines, and successfully executing the **SVFinder** intelligent decision-support platform (comprising a FastAPI backend engine coupled with a Laravel MVC frontend web application).

## 1. System Requirements & Prerequisites

Before initiating environment configuration inside Visual Studio Code (or alternative preferred integrated development environments), verify that both hardware specifications and foundational global software runtimes comply with the metrics detailed below.

### 1.1 Hardware Specifications

- **Processor (CPU):** Intel Core i5 (or AMD ryzen equivalent architecture) or higher.
- **Memory (RAM):** Minimum 8GB system physical memory capability.
- **Storage Workspace:** Allocation of at least 500MB available persistent local storage space (handling environment packages, structural dependencies, and local vector directory nodes).

### 1.2 Fundamental Software Runtimes

The following runtimes must be provisioned and accessible globally within your machine's system environment paths:

- **Python Runtime:** Version 3.8 or superior.
- **PHP Engine:** Version 8.x (mandated to support Laravel architectural framework execution).
- **Composer Dependency Manager:** Provisioned for backend PHP routing libraries.
- **Node.js & npm Core:** Installed to orchestrate and compile client-side interactive assets.

## 2. IDE & Local Environment Setup

Execute this sequential series of steps within your terminal framework to cleanly construct an isolated environment ecosystem.

### Step 1: Workspace Initialization

Launch Visual Studio Code, navigate to `File > Open Folder`, and select the master **SVFinder** repository directory path. Initialize a native terminal panel (`Ctrl + ``).

## Step 2: Python Virtual Environment Isolation

Generate a clean, completely isolated local virtual directory environment ( `.venv` ) inside your active project root workspace to guard against host python library conflicts:

```
# Initialize the localized virtual environment folder
python -m venv .venv

# Active execution path invocation (Windows Architecture)
.venv\Scripts ctivate

# Active execution path invocation (macOS / Linux Environments)
source .venv/bin/activate
```

## Step 3: Automated Dependency Batch Installation

Verify that the standardized software manifesto file `requirements.txt` is present within your active project workspace. It must contain the following core dependencies:

```
fastapi
uvicorn
requests
chromadb
python-dotenv
sentence-transformers
langchain
langchain-huggingface
langchain-core
pydantic
```

With your active terminal successfully tracking inside the ( `.venv` ) context wrapper, fire the following batch installation manager command:

```
pip install -r requirements.txt
```

## Step 4: Environment Key Configuration (.env)

Establish a localized secret metadata file named exactly `.env` inside the parent project directory. Populate this matrix with valid Hugging Face API tokens required to authorize communication with the remote Large Language Model endpoints:

```
# Hugging Face Application Interface Secrets
HF_API_KEY="your_verified_huggingface_platform_token_here"
HF_TEMP_KEY="your_secondary_or_backup_hf_token_here"
```

### Step 5: Frontend Framework Asset Setup

Orchestrate the initialization of packages and script frameworks mapped to the user interface tier by firing the following structural manager tools:

```
# Resolve and pull structural Laravel components via Composer
composer install

# Fetch and link client-side styling modules and scripts via npm
npm install
```

## 3. Application Execution Workflow (Dual-Server Launch)

To run the SVFinder system smoothly, you must execute the Python backend and Laravel frontend simultaneously using split terminals inside your main project directory.

### 3.1 Terminal Session A: FastAPI Engine Core

Ensure the Python virtual environment ( `.venv` ) is fully engaged. Execute the ASGI production server wrapper script to instantiate the active RAG pipelines and API routes:

```
uvicorn fyp-backend.main:app --reload --port 8000
```

- `fyp-backend.main:app` indicates the main application entry point inside your backend folder.
- The `--reload` directive monitors code modifications locally and hot-swaps runtimes instantly.
- The internal communication engine default assigns to network **Port 8000**.

### 3.2 Terminal Session B: Laravel Client Web Application (Split Window)

Instantiate a secondary concurrent terminal connection window within the workspace environment, descend directly into the designated user interface folder path, and raise the PHP application interface layer:

```
cd fyp-frontend
php artisan serve --port 8080
```

- The web dashboard will host locally on network **Port 8080** (accessible via `http://127.0.0.1:8080` ).
- Launch any standard modern web browser engine and connect to the generated local address to interact with the platform interface.

## 4. Technical Pipeline Process Overview

Once an input payload is processed via the client dashboard, the internal LangChain orchestration handles execution across a strict two-stage pipeline layer:

| Execution Stage         | Engine Components Involved                               | Core Processing Activity                                                            |
|-------------------------|----------------------------------------------------------|-------------------------------------------------------------------------------------|
| 1. Query Vectorization  | Sentence-Transformers ( <code>all-mpnet-base-v2</code> ) | Converts user input text queries into high-dimensional numerical vectors.           |
| 2. Contextual Retrieval | ChromaDB Local Vector DB Instance                        | Runs vector similarity lookups to retrieve top-3 matching supervisor contexts.      |
| 3. Generative Reasoning | LangChain + Hugging Face Base LLM                        | Processes the retrieved supervisor meta-context to generate natural justifications. |